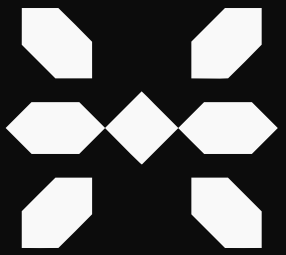
HOOX · PYNE · DOCUMENTATION



EDITOR INTEGRATORS · IDE AUTHORS

LANGUAGE SERVER MANUAL

LSP features, builtin metadata, VS Code, editor clients

Open-source Pine Script evaluation framework

CONTENTS

01 Overview

02 Architecture

03 Diagnostics

04 Completion

05 Hover

06 Navigation

07 Formatting

08 Semantic Tokens

09 Inlay Hints

10 Builtin Metadata

11 Vscode Extension

12 Clients

OVERVIEW

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "Language Server" description: "PYNE Language Server Protocol — diagnostics, completion, hover, formatting for .pyne / .pine and editor clients."

LANGUAGE SERVER

The PYNE **Language Server** is the editor-facing half of the evaluation stack: a pygls process that speaks LSP over STDIO (or TCP in tooling), parses Pine with the same ANTLR4 → ASDL pipeline as the runtime, and never requires a proprietary chart host. First-class file associations include `.pyne` (product extension) plus `.pine` / `.pinev5` / `.pinev6` / `.pinescript`.

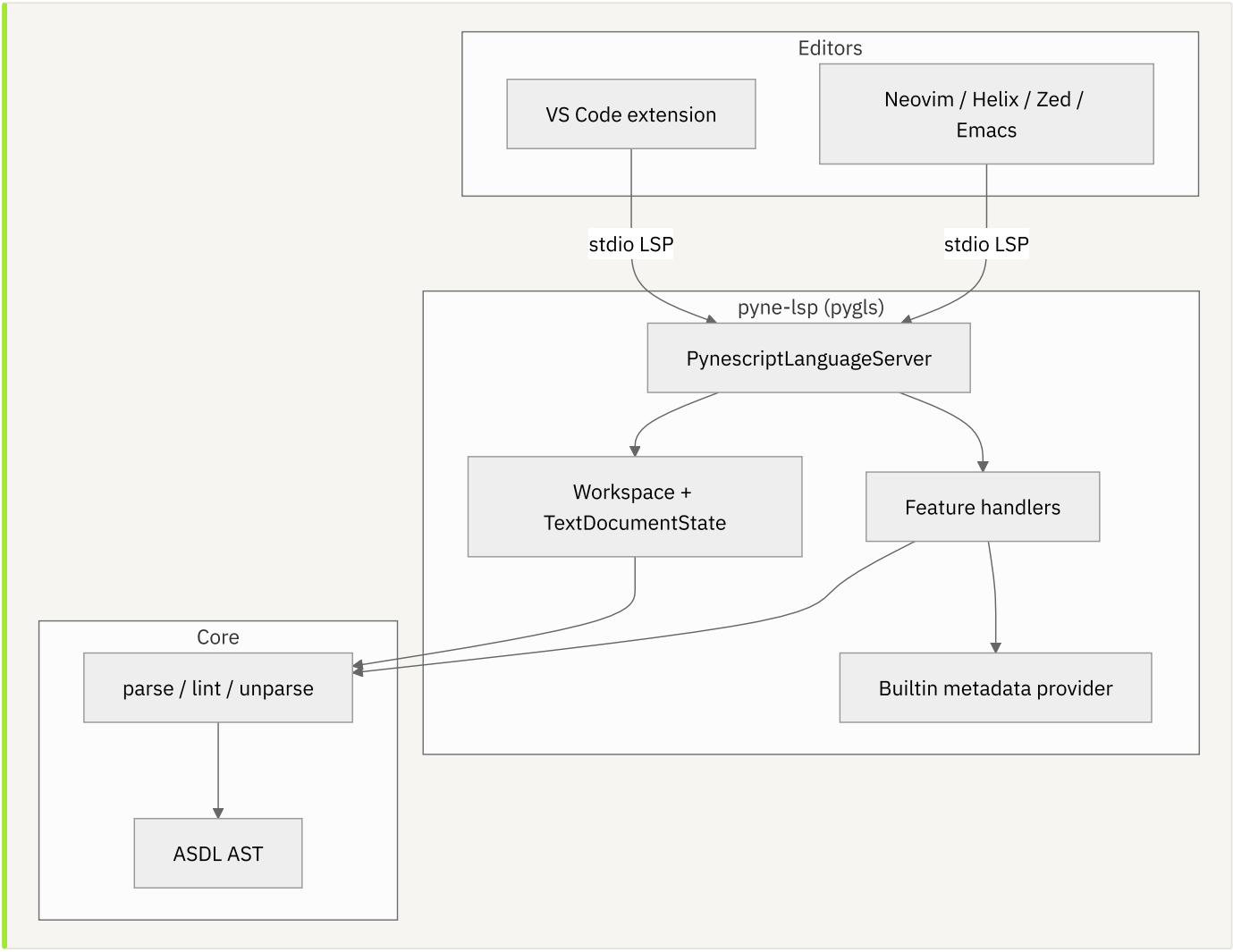
Abstract

Editors want *static* intelligence at typing cadence. Evaluation over OHLCV is *dynamic* and bar-loop expensive. PYNE therefore splits the stack:

Process	Cadence	Shared substrate
LSP (`pyne-lsp`)	keystroke / open / save	parse, lint, unparse, builtin metadata
Pro API (`backend.app`)	request / batch	parse, evaluate, plots / events / alerts
CLI / library	batch / embed	same AST + evaluator

The LSP publishes diagnostics from the linter and parse errors, completes and hovers from Fernet-ready builtin metadata (plus keywords and user enums), navigates user symbols via AST visitors, emits semantic tokens from the cached tree, and formats by round-tripping through the unparser. Feature modules live under `src/pynescript/langserver/features/` ; the VS Code extension (`hoox-sh.pyne` **0.4.4**) and `clients/` configs are thin protocol hosts — VS Code maps `.pyne` **first**, then legacy Pine suffixes, to language id `pinescript` . The stdio image is **`ghcr.io/hoox-sh/pyne/lsp`** .

Conceptual model



Invariant: open/change/close always re-parse and re-lint the in-memory buffer. Features that need an AST re-parse on demand if the workspace cache is missing source.

Interface surface

Capability	LSP method(s)	Module
Sync	`textDocument/didOpen	didChange
Diagnostics (push + pull)	`publishDiagnostics`, `textDocument/diagnostic`	`workspace.py`, `features/diagnostics.py`
Completion (+ resolve)	`textDocument/completion`, `completionItem/resolve`	`features/completion.py`
Hover	`textDocument/hover`	`features/hover.py`
Definition / references	`textDocument/definition`, `textDocument/references`	`definitions.py`, `references.py`
Symbols	`textDocument/documentSymbol`, `workspace/symbol`	`symbols.py`, `server.py`
Formatting	`textDocument/formatting`, `rangeFormatting`	`features/formatting.py`
Inlay hints	`textDocument/inlayHint`	`features/inlay_hints.py`
Semantic tokens	`textDocument/semanticTokens/full`	`features/semantic_tokens.py`

Declared capabilities: `src/pynescript/langserver/config.py` (`get_server_capabilities`).

Internals

Path	Role
`src/pynescript/langserver/server.py`	`PynescriptLanguageServer`, method registration
`src/pynescript/langserver/workspace.py`	Document map, incremental edits, parse+lint
`src/pynescript/langserver/config.py`	ServerCapabilities
`src/pynescript/langserver/features/*`	Per-method handlers
`src/pynescript/langserver/providers/*`	Metadata + completion builders
`src/pynescript/langserver/__main__.py`	STDIO entry (`pyne-lsp` / alias `pynescript-lsp`)
`vscode-extension/`	Language client + TextMate grammar (`hoox-sh.pyne` 0.4.4)
`clients/`	Neovim, Zed, Emacs snippets (Helix / Sublime are documented, not files)

Console scripts are separate: `pyne` (Click CLI; alias `pynescript`) vs `pyne-lsp` (pygls; alias `pynescript-lsp`). Do not conflate them — see architecture.

Tracks in this tab

1. Architecture — server lifecycle, workspace, capabilities
2. Diagnostics through Inlay hints — feature manuals
3. Builtin metadata — generation + Fernet `.enc` / `CRYPTO_KEY`
4. VS Code extension — client packaging
5. Clients — Neovim, Zed, Emacs, Helix, Sublime

See also

- Pro API — HTTP evaluate path sharing the same AST
- Evaluate contract
- Linter
- Unparser
- Editors guide (end user)

ARCHITECTURE

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

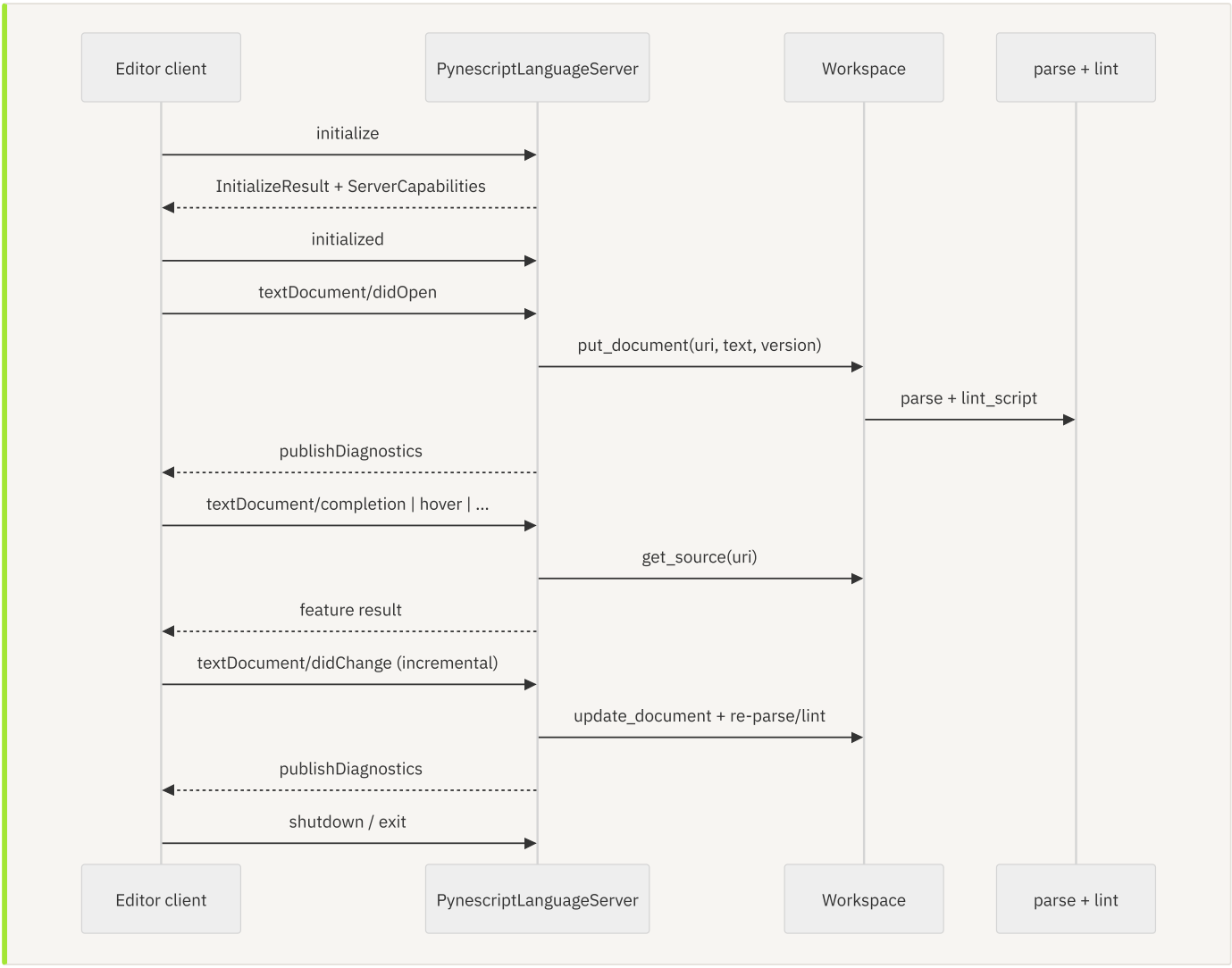
title: "LSP Architecture" description: "PynescriptLanguageServer lifecycle, Workspace document model, capability declaration, and STDIO transport."

LSP ARCHITECTURE

Abstract

The language server is a single-process pygls application. It owns a `Workspace` of open `TextDocumentState` records, re-parses on every mutation, and dispatches LSP methods to pure-ish feature handlers that take `(params, source)` rather than holding server globals. Capabilities are declared once at `initialize`; transport defaults to STDIO for editor integration.

Conceptual model



Interface surface

Process entry

```
# Editable install
pip install -e ".[lsp]"
python -m pynescript.langserver   # same as pyne-lsp / pynescript-lsp
make run-lsp
```

Entry: `src/pynescript/langserver/__main__.py` constructs `PynescriptLanguageServer()` and calls `server.start_io()` (STDIO JSON-RPC).

`pyproject.toml` registers:

```
pyne-lsp = "pynescript.langserver.__main__:main"
pynescript-lsp = "pynescript.langserver.__main__:main"   # alias
```

Server class

`PynescriptLanguageServer` (`server.py`):

- Subclasses `pygls.lsp.server.LanguageServer` with `name="Pynescript"` and `version=__version__` from `src/pynescript/__about__.py` (**0.6.7**).
- Instantiates `self.pine_workspace = Workspace()`.
- Registers handlers in `setup_method_handlers()` via `@self.feature(...)`.

Document sync

Event	Behavior
`didOpen`	`put_document` → parse/lint → push diagnostics
`didChange`	Incremental or full-text apply → re-parse/lint → push diagnostics
`didClose`	Remove document; publish empty diagnostic list
`didSave`	Re-publish diagnostics for current buffer

Sync options (`config.get_server_capabilities`):

- `TextDocumentSyncKind.Incremental`
- `open_close=True`
- `save` with `include_text=True`
- `will_save` / `will_save_wait_until` off

Capability set (declared)

From `config.py`:

- Diagnostic provider (`identifier="pynescript-diagnostics"`, `workspace_diagnostics=False` in options; workspace pull handler still exists)
- Completion: trigger `"."`, `resolve_provider=True`
- Hover, definition, references
- Document + workspace symbols
- Full + range formatting
- Code action (Convert to Pine v6) + executeCommand `pynescript.convertToV6`
- Inlay hints (`resolve_provider=False`)
- Semantic tokens full (legend of standard token types/modifiers; range=false)
- Signature help is **not** advertised.

File filters (`get_filter_options`): `*.pine`, `*.pinev5`, `*.pinev6` under language id `pinescript`. **`.pyne` is not in this helper** — VS Code still maps it via the extension contribution (`extensions: .pyne` first).

Internals

Workspace

`Workspace` (`workspace.py`) maps `uri → TextDocumentState`:

```
TextDocumentState
  uri, source, version
  ast | None
  diagnostics: list[LintWarning]
  parse_error, parse_error_line
```

`_parse_and_lint`:

1. `parse(source, filename=uri)` — on success, `lint_script(source, filename=uri)`.
2. On exception: clear AST, store `parse_error` string, extract line via regex `line[:\s]+(\d+)`.

Incremental edits: `_apply_text_edit` splits on `\n`, pads the line list when the range sits at/past EOF (append without a trailing newline), clamps columns, then replaces the range. Out-of-range positions no longer leave a stale buffer. Whole-document change events replace `source` wholesale. Identical text after apply skips re-parse/lint (version still updates).

Diagnostics conversion (`_lint_warnings_to_diagnostics`):

- Map severity strings → `DiagnosticSeverity`.
- Source tag `"PineScript"`, code from lint rule.
- Append synthetic **E001** error for parse failures at the extracted line.

Feature dispatch pattern

Handlers are functions, not server methods, e.g.:

```
@self.feature(lsp.TEXT_DOCUMENT_COMPLETION)
def text_completion(params):
    source = self.pine_workspace.get_source(params.text_document.uri)
    return completion_feature.handle_completion(params, source)
```

Definition / references / symbols / hover / completion / inlay / semantic tokens receive the **workspace-cached AST** (`tree=doc.ast`) so they skip a redundant parse. Pass `tree=None` when the last parse failed. Omit `tree` only in isolated tests (handler then parses from `source`).

Pull diagnostics

- `textDocument/diagnostic` → `RelatedFullDocumentDiagnosticReport` with `result_id=f"{uri}-{version}"`.
- `workspace/diagnostic` → one full report per open document from `get_all_diagnostics()`.

Workspace symbols

`workspace/symbol` walks each open document's AST via `_collect_workspace_symbols`: `FunctionDef` → Function, `TypeDef` → Class, `EnumDef` → Enum, `Assign` to `Name` → Variable. Filter is case-insensitive substring on `params.query`.

Invariants and edge cases

1. **Parse failure is non-fatal for the process.** AST-dependent features return `None` or `[]`; diagnostics still surface E001.
2. **Source of truth is the open buffer**, not disk. Save only re-publishes; it does not re-read files.

3. **Incremental sync pads/clamps rather than dropping the edit.** Ranges at EOF grow the line list; columns are clamped to the current line length.
4. **Version** is stored and used in pull-diagnostic `result_id`; the server does not reject out-of-order versions itself.
5. **Nuitka onefile** embeds providers data and may rely on encrypted metadata — see builtin metadata. Architecture of handlers is identical to the pure-Python path.

Worked example — minimal initialize

Client → server (conceptual):

```
{
  "method": "initialize",
  "params": {
    "capabilities": {},
    "clientInfo": { "name": "example", "version": "1" }
  }
}
```

Server returns `InitializeResult` with `serverInfo.name = "Pynescript Language Server"` and the full `ServerCapabilities` table from `get_server_capabilities()`.

Failure modes

Symptom	Likely cause
No diagnostics on open	Client did not send `didOpen` or language id is not `pinescript`
Completions empty	Metadata file missing/undecryptable; see builtin metadata
Format no-ops	Parse error (handler returns `[]`) or unparsed text equals source
Extension can't spawn server	`pynescript-lsp` not on `PATH`; check VS Code extension

See also

- Diagnostics
- Builtin metadata
- VS Code extension
- Nuitka build

DIAGNOSTICS

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

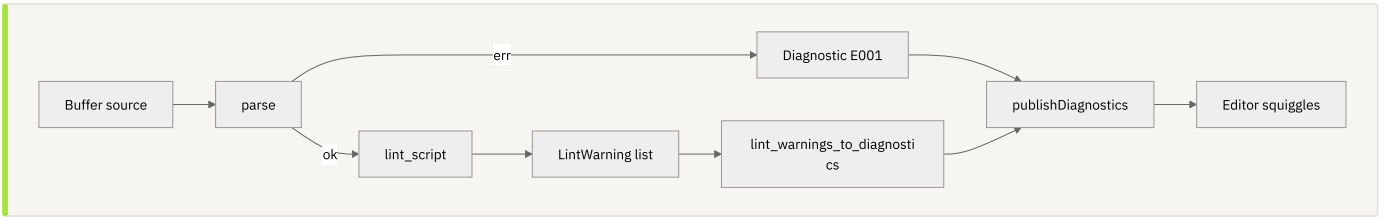
title: "Diagnostics" description: "Push and pull diagnostics: parse errors, lint rules, severity mapping, and code-action stubs."

DIAGNOSTICS

Abstract

Diagnostics are the primary static feedback channel. The workspace re-parses and re-lints on every open/change/save, converts `LintWarning` objects (and parse failures) into `lsp.Diagnostic`, then **pushes** them with `textDocument/publishDiagnostics`. Clients that support LSP 3.16+ pull diagnostics can also request `textDocument/diagnostic` or `workspace/diagnostic`.

Conceptual model



Interface surface

Push (always on)

Triggered from `server.py` on `didOpen`, `didChange`, `didSave`. Close clears diagnostics (`items=[]`).

Pull

Method	Response
`textDocument/diagnostic`	`RelatedFullDocumentDiagnosticReport` with `kind=full`, `result_id="{uri}-{version}"`
`workspace/diagnostic`	One `WorkspaceFullDocumentDiagnosticReport` per open URI

Capability: `diagnostic_provider` with `identifier="pynescript-diagnostics"`, `inter_file_dependencies=False`.

Diagnostic shape

Field	Value
`range`	0-based line; column from warning or 0; end spans remainder of line (capped)
`severity`	error / warning / information / hint
`message`	Human-readable lint or parse message
`source`	`"PineScript"`
`code`	Lint code (e.g. `W001`) or `E001` for parse errors

The richer conversion in `features/diagnostics.py` also supports:

- `code_description` hrefs for codes starting with `E` / `W` (docs URLs)
- `tags`: `W001` → Unnecessary, `W002` → Deprecated

Push and pull both call `Workspace._lint_warnings_to_diagnostics`, which delegates lint rows to `features/diagnostics.lint_warnings_to_diagnostics` (noise filters + tags) and then appends synthetic **E001** for parse failures.

Internals

Path	Role
`src/pynescript/langserver/workspace.py`	`_parse_and_lint`, E001 append, incremental edits
`src/pynescript/langserver/features/diagnostics.py`	Lint conversion, C001 noise filter, tags, `create_quick_fix`
`src/pynescript/ast/linter.py`	Rule engine producing `LintWarning`

Severity map

```
error        → DiagnosticSeverity.Error
warning      → Warning
info|information → Information
hint         → Hint
(default)    → Warning
```

Parse errors

On exception from `parse`:

- `ast = None`, `diagnostics = []` (lint skipped)
- Message stored as `parse_error`
- Synthetic diagnostic code **E001**, severity Error, line from regex on the exception text

Quick fixes (feature module)

`create_quick_fix` (not yet fully wired through `workspace/executeCommand` / codeAction handlers in `server.py`):

- **W001** → insert `//@version=6\n` at document start
- **C002** (long line) → suggest format document command

Capability advertises QuickFix / Refactor / SourceOrganizeImports kinds for future wiring.

Invariants and edge cases

1. **Lint does not run when parse fails** — only E001 is shown.
2. Warnings without a line number are dropped (`None` conversion).
3. End column heuristic uses the remainder of the line and clamps to 2000 characters — not a precise token span.
4. `inter_file_dependencies=False`: no cross-file analysis; each URI is independent.
5. Push and pull should agree for a given buffer version; both read the same `TextDocumentState`.
6. **Noise filters:** `C001` on names that already lack `_` (camelCase `fastMA`) is dropped. The `C003` block-`if` noise filter was removed with the rule (retired 0.6.0).

Worked example

Source:

```
indicator("x")
plot(close)
```

If the linter emits a missing-version warning (code `W001`), the client receives a Warning diagnostic on the relevant line with `source: "PineScript"` and may offer the version-header quick fix when code actions are connected.

On a hard parse failure at line 3, expect:

```
{
  "severity": 1,
  "code": "E001",
  "source": "PineScript",
  "message": "<exception text>"
}
```

Failure modes

Symptom	Cause
Stale squiggles after edit	Client using full-document sync incorrectly, or version mismatch
Empty diagnostics on broken file	Unexpected: parse errors should still emit E001 — check client filter on `source`
No workspace pull results	Only **open** documents are tracked; closed files are absent

See also

- Linter (core)
- Error model
- Architecture

COMPLETION

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

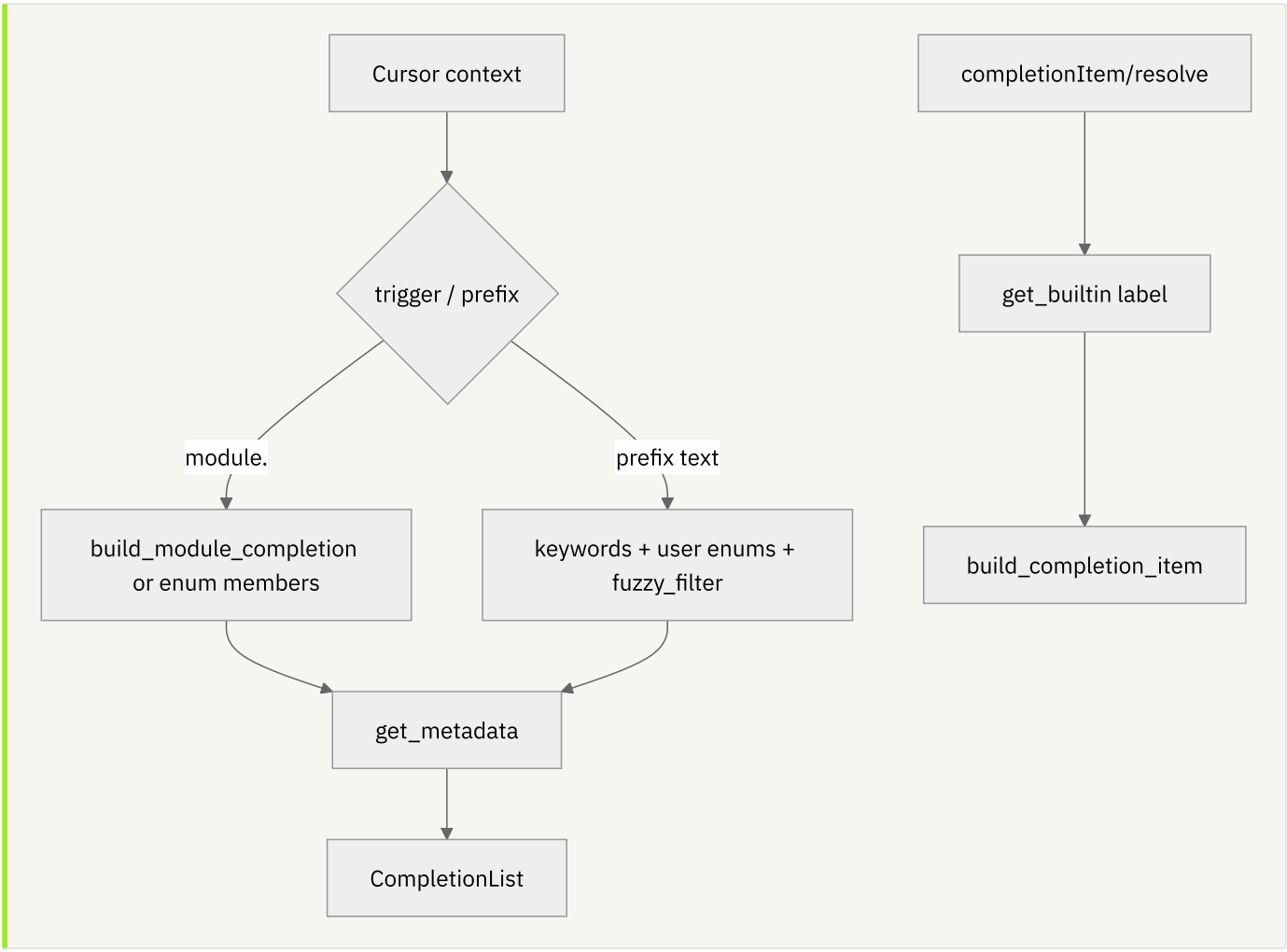
title: "Completion" description: "textDocument/completion and completionItem/resolve from builtin metadata, module-dot triggers, and fuzzy filter."

COMPLETION

Abstract

Completion is catalog-driven, not evaluator-driven. The server loads builtin metadata (plaintext JSON in development, Fernet-encrypted in compiled binaries), then also offers **Pine keywords** (`PINE_KEYWORDS`) and **user** `enum` **types / members** collected from the workspace AST. Resolve re-hydrates documentation for a single item.

Conceptual model



Interface surface

Method	Capability
`textDocument/completion`	`trigger_characters=["."]`, returns `CompletionList`
`completionItem/resolve`	`resolve_provider=True`

Handler: `features/completion.py` → `handle_completion` / `handle_completion_resolve`.

Trigger logic

1. Compute `text_before_cursor` on the current line.
2. `get_trigger_char` — if the previous character is `.` (or `(`, `,`, space), treat as trigger.
3. Collect user enums from the cached AST (`collect_user_enums`).
4. If the last token contains `.` (e.g. `ta.` or `Side.`):
   - user-enum prefix → `build_enum_member_completion`
   - else `build_module_completion(module)` for catalog namespaces
5. Else merge `build_keyword_items`, `build_enum_name_items`, and `build_completion_list(prefix=prefix)`.

Word boundaries for Pine identifiers: `[a-zA-Z_][a-zA-Z0-9_.]*` (`protocol/utils.py`).

Item fields

Built by `providers/completion_items.py`:

Field	Source
`label`	metadata `label` (e.g. `ta.sma`)
`kind`	`CompletionItemKind.Function`
`detail`	signature / detail string
`documentation`	Markdown from metadata
`insert_text`	Snippet if `${...}` present, else plain label
`insert_text_format`	Snippet or PlainText
`filter_text`	dotted parts + brief
`sort_text`	modules first (`\x01…`), root second (`\x02…`)

Category headers may appear as Folder-kind items with empty insert text and labels like `--- Technical Analysis (ta.*) (N) ---`.

Internals

Path	Role
`features/completion.py`	Request context + dispatch
`providers/completion_items.py`	List / item / module builders
`providers/builtin_metadata.py`	Load cache, `fuzzy_filter`, `get_builtin`
`protocol/utils.py`	Word + trigger helpers

Fuzzy filter scores

`fuzzy_filter(query, items, limit=50)`:

Match	Score
Exact label	1000
Prefix	500
Substring in label	100
Category	50
Brief	25

Results sorted by score descending, capped at `limit`.

Resolve

`handle_completion_resolve` looks up `params.label` in metadata; if found, rebuilds a full `CompletionItem`. Unknown labels return unchanged.

Invariants and edge cases

1. **No local function / variable completion yet** — user **enums** and **keywords** are included; local `myFunc` is not, unless it appears in metadata.
2. Empty metadata dict yields an empty list (failed decrypt / missing files).
3. Module completion uses `startswith(module + ".")` — nested namespaces beyond one dot still work if labels are fully qualified.
4. Snippets are generated at metadata build time (`scripts/generate_builtin_metadata.py`); incompleteness of placeholders is a generator concern, not the LSP handler.

Worked example

User types `ta.` → module completion for all `ta.*` labels.

User types `sma` without a module → fuzzy filter may return `ta.sma` among others (substring score).

Resolve on `ta.sma` reloads full documentation markup for the detail pane.

Failure modes

Symptom	Cause
Zero items always	Metadata not loaded — check builtin metadata
Dot trigger ignored	Client did not set completion trigger characters from server capabilities
Snippets inserted raw	Client disabled snippet support; extension setting `pynescript.completion.snippets`

See also

- Hover — shares metadata
- Builtin metadata
- Inlay hints — uses return types from `detail`

HOVER

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

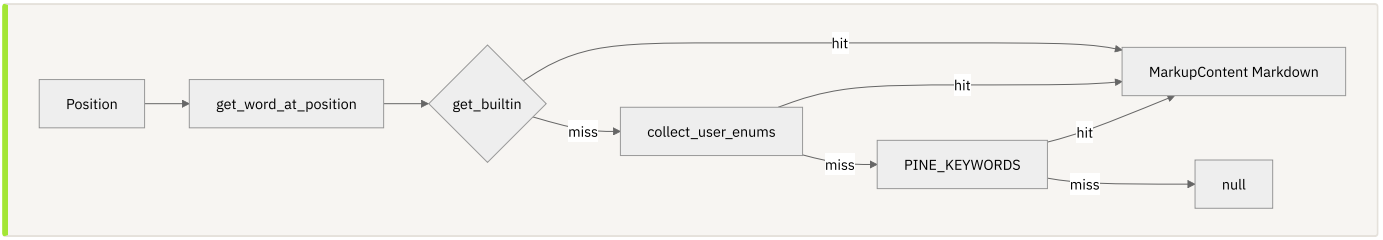
title: "Hover" description: "textDocument/hover — Markdown documentation cards for Pine builtins from metadata."

HOVER

Abstract

Hover answers “what is this identifier?” for **builtins**, then **user enums**, then **keywords**. The handler extracts the word under the cursor (including dotted forms), looks up metadata / enum members / `PINE_KEYWORDS`, and returns a Markdown card. Local functions and variables still return `null` (use definition).

Conceptual model



Interface surface

- Method: `textDocument/hover`
- Capability: `hover_provider=True`
- Handler: `features/hover.py` → `handle_hover`

Lookup order

1. Exact word at cursor via `get_word_at_position` (identifiers may include `.`).
2. Builtin lookup: full word, then leaf after `.`, then `module.word` if the preceding token ends with `.`.
3. Else user-enum type or `Enum.member` from the cached AST.
4. Else keyword brief from `PINE_KEYWORDS`.
5. Build hover with range covering `[start, end)` of the word on the line.

Card structure

```
```pinescript
{detail}
```

{brief}

{first documentation paragraph, ≤500 chars}

Example: … **See also:** `ta.ema`, …
```

```
Examples and related maps are **hardcoded** for a small high-value set (`ta.sma`, `ta.ema`, `ta.rsi

External TradingView doc links are intentionally omitted (`_build_docs_link` returns empty) to keep

## Internals

| Path | Role |
| --- | --- |
| `features/hover.py` | Word resolution + Markdown assembly |
| `providers/builtin_metadata.py` | `get_builtin` |
| `protocol/utils.py` | Word extraction |

## Invariants and edge cases

1. **No source → null.** Missing buffer aborts early.
2. **Line OOB → null.** Cursor past last line is ignored.
3. **Empty word → null.** Whitespace / punctuation yields no hover.
4. **User functions / locals / UDTs** are not inferred for hover (enums are). Use [definition](/pyn
5. Documentation truncation is first-paragraph / 500-char only — full docs live in metadata and com

## Worked example

Cursor on `sma` in `plot(ta.sma(close, 14))`:

- Word may be `ta.sma` if the dotted identifier is one match, or `sma` with module recovery from `t
- Response `contents.kind = markdown` with fence of the metadata `detail` line.

## Failure modes

| Symptom | Cause |
| --- | --- |
| Hover never appears | Metadata empty or word not in catalog |
| Wrong symbol | Identifier regex swallowed neighboring text — rare with `_.` pattern |
| Stale docs after adding builtin | Regenerate metadata — [builtin metadata](/pyne/docs/lsp/builtin

## See also

- [Completion](/pyne/docs/lsp/features/completion)
- [Builtin metadata](/pyne/docs/lsp/builtin-metadata)
- [Navigation](/pyne/docs/lsp/features/navigation)
```

NAVIGATION

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

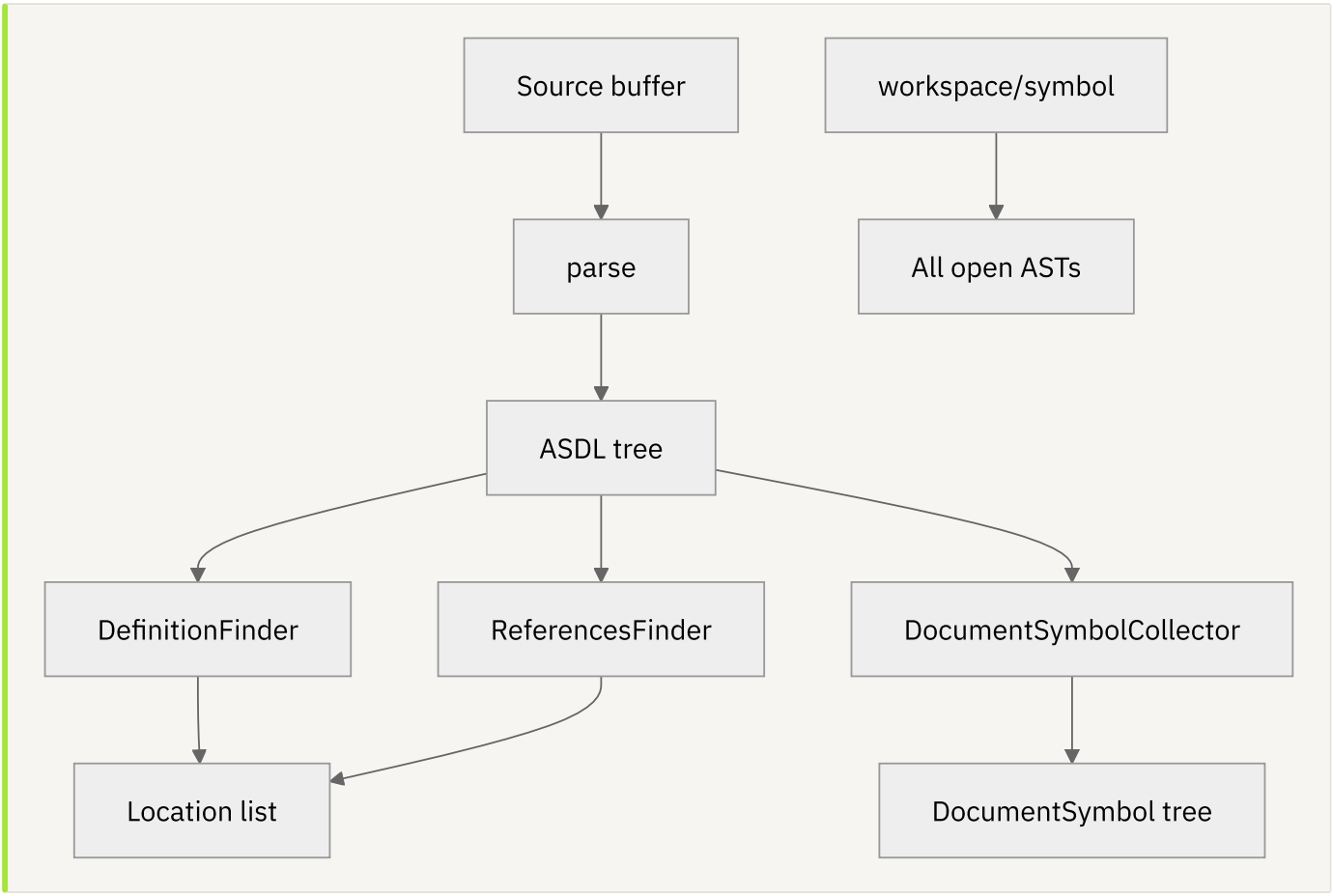
title: "Navigation" description: "Go-to-definition, find-references, document outline, and workspace symbol search over the ASDL AST."

NAVIGATION

Abstract

Navigation features operate on **user AST symbols**, not the builtin catalog. Definition and references walk a freshly parsed tree with `NodeVisitor` subclasses; document symbols build a hierarchical outline; workspace symbols scan all **open** documents. Locations currently anchor to line starts (coarse ranges) — enough for jump lists, not character-precise rename.

Conceptual model



Interface surface

Method	Handler	Returns
`textDocument/definition`	`definitions.handle_definition`	`list[Location] | None`
`textDocument/references`	`references.handle_references`	`list[Location]` (possibly empty)
`textDocument/documentSymbol`	`symbols.handle_document_symbols`	`list[DocumentSymbol]`
`workspace/symbol`	`server._collect_workspace_symbols`	`list[SymbolInformation]`

Capabilities: `definition_provider`, `references_provider`, document + workspace symbol options with work-done progress flags.

Definition (`DefinitionFinder`)

Matches target word against:

- `FunctionDef.name`
- `TypeDef.name`
- `Assign` targets (`Name` or nested names)

Ignores mere call sites (`visit_Call` returns early when the callee name matches).

References (`ReferencesFinder`)

- `Name` with Load / Store contexts

- Function and type definitions when `include_declaration` is true
- Call sites of bare `Name` callees

`params.context.include_declaration` controls whether declarations appear.

Document symbols

Hierarchical:

AST	SymbolKind	Children
`FunctionDef`	Function	Variables assigned inside body
`TypeDef`	Class	Fields (`Assign` targets)
`EnumDef`	Enum	Members
Top-level `Assign`	Variable	—

Function `detail` is a coarse `function name()` string; assignments may show callee module attribute as detail when RHS is a call.

Workspace symbols

Flat `SymbolInformation` for functions, types, and assigned names across open buffers; filtered by lowercase query substring.

Internals

Path	Role
`features/definitions.py`	Go-to-definition visitor
`features/references.py`	Find-all-references visitor
`features/symbols.py`	Document outline
`server.py`	Workspace symbol aggregation

Handlers accept the **workspace-cached AST** (`tree=doc.ast` from `server.py`). They re-parse from `source` only when `tree` is omitted (tests / isolated calls). Pass `tree=None` after a failed parse so the handler does not retry.

Invariants and edge cases

1. **Parse failure → empty / null** (definition `None` , others `[]`).
2. **Builtin names** have no definition in-file; clients should not expect `ta.sma` to resolve to a library file URI.
3. **Ranges are line-coarse** for many locations (`character=0`); selection ranges for document symbols use `col_offset` when present.
4. **Scope is naive** — same identifier in nested functions may collect multiple definitions; no shadowing analysis.
5. **Closed documents** disappear from workspace symbol search.

Worked example

```
//@version=6
indicator("nav")
length = 14
mySma(src, len) =>
    ta.sma(src, len)
v = mySma(close, length)
```

- Definition on `mySma` at the call site → function line.
- References on `length` → assign + call argument (and declaration if included).
- Outline → `length` variable, `mySma` function (with locals), `v` variable.

Failure modes

Symptom	Cause
Jump lands column 0	Expected with current Location encoding
Missing references inside function when targeting another function	`visit_FunctionDef` only walks body when the def name **differs** from target — intentional for declaration handling; calls inside same-named functions are a known edge
Empty outline	Parse error or empty source

See also

- Architecture
- Hover — builtins vs user symbols
- Diagnostics

FORMATTING

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

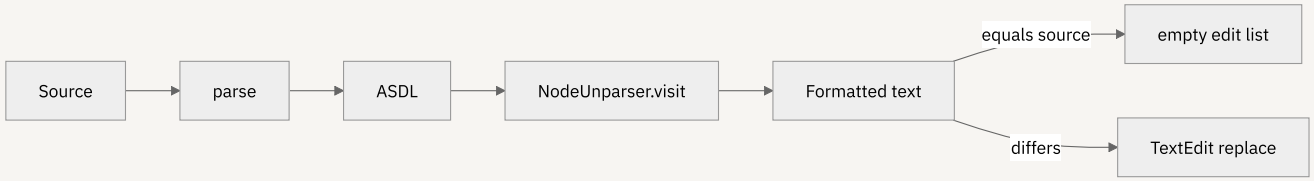
title: "Formatting" description: "Document and range formatting via AST parse → NodeUnparser round-trip."

FORMATTING

Abstract

Formatting is **canonical unparse**, not a style linter. The handler parses the buffer, runs `NodeUnparser`, and if the result differs from the source, replaces the document (or the requested line range) with a single `TextEdit`. There is no prettier-like option matrix; `FormattingOptions` from the client are currently unused.

Conceptual model



Interface surface

Method	Handler
`textDocument/formatting`	`handle_formatting`
`textDocument/rangeFormatting`	`handle_range_formatting`

Capabilities: both providers `True` in `config.py`.

Full document

1. Parse with filename `"<format>"`.
2. `formatted = NodeUnparser().visit(tree)`.
3. If identical to source → `[]`.
4. Else one `TextEdit` spanning `(0,0)` → last line/col with `new_text=formatted`.

Range

1. Unparse the **entire** document.
2. Slice **formatted** lines and **source** lines by the client range's start/end line indices.
3. If the in-range slices match → `[]`.
4. Else replace `params.range` with the formatted line slice.

This is line-aligned, not AST-node-aligned: partial mid-line ranges may produce surprising splices when unparse changes line structure.

Internals

Path	Role
`features/formatting.py`	LSP handlers
`src/pynescript/ast/helper.py`	`parse`
`src/pynescript/ast/unparser.py`	`NodeUnparser`

VS Code command `pynescript.formatDocument` delegates to `editor.action.formatDocument`, which hits this provider when the language client is active.

Invariants and edge cases

1. **Parse failure** → `[]` (no partial format; silent no-op from the client's perspective).

2. **Idempotence goal:** format twice should stabilize if unparse is canonical; any non-idempotent unparse is a core bug, not an LSP bug.
3. **Comments / trivia:** fidelity is whatever the unparser preserves — not a full CST pretty-printer.
4. **Range formatting** still requires a full successful parse of the whole file.
5. Tab size / insert spaces from `FormattingOptions` are ignored today.

Worked example

Before:

```
//@version=6
indicator("x")
plot(  close  )
```

After full format (illustrative — exact whitespace follows `NodeUnparser`):

```
//@version=6
indicator("x")
plot(close)
```

If source already matches unparse output, the server returns an empty edit list and the editor leaves the buffer alone.

Failure modes

Symptom	Cause
Format does nothing on broken syntax	Expected: exception → `[]`
Range format rewrites wrong lines	Unparse shifted line count; prefer full-document format
Client “format on save” loops	Unparse not idempotent — fix unparser, not client

See also

- Unparser
- Architecture
- VS Code extension

SEMANTIC TOKENS

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

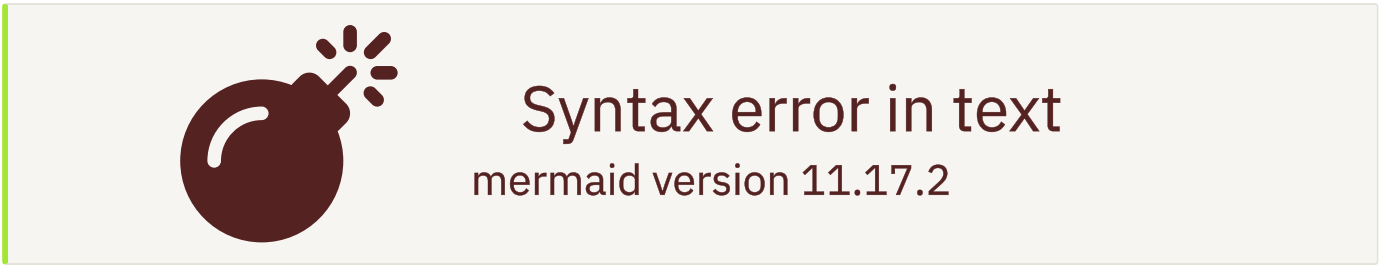
title: "Semantic Tokens" description: "textDocument/semanticTokens/full — AST visitor, custom legend, and LSP delta encoding."

SEMANTIC TOKENS

Abstract

Semantic tokens let the server paint identifiers with roles richer than TextMate scopes (library namespaces vs user variables, function defs vs properties). PYNE advertises a **full** semantic-tokens provider with a **short custom legend** (not the full LSP standard set) and implements `textDocument/semanticTokens/full` by walking the workspace AST.

Conceptual model



Interface surface

Capability (`config.py` → `semantic_tokens_provider`):

```
legend.token_types (index order is contract):
  namespace, type, class, function, method, variable,
  parameter, property, keyword, string, number, operator, comment

legend.token_modifiers (bit 0 = declaration):
  declaration, definition, readonly, defaultLibrary

range: false
full: true
```

Handler: `features/semantic_tokens.py` → `handle_semantic_tokens(params, source, tree=…)` .

Response type: `lsp.SemanticTokens(data=[...])` — LSP-encoded five-tuples as a flat `int` array (relative line/character deltas).

What is emitted

AST	Token type	Modifiers
`FunctionDef.name`	function	definition | declaration
`TypeDef.name`	class	definition | declaration
`EnumDef.name`	type	definition | declaration
`Assign` to `Name`	variable	declaration
`Attribute` whose value is a builtin namespace (`ta` , `math` , `strategy` , …)	namespace + method	defaultLibrary (+ readonly on ns)
Other `Attribute.attr`	property	—

Builtin namespace set (`_BUILTIN_NS`): `ta` , `math` , `str` , `array` , `matrix` , `map` , `strategy` , `request` , `input` , `color` , `line` , `label` , `box` , `table` , `polyline` , `log` , `ticker` , `timeframe` , `chart` , `runtime` , `syminfo` , `barstate` , `session` , `time` .

Internals

Path	Role
`features/semantic_tokens.py`	Collector + `_encode`
`config.py`	Legend + capability — **indices must match** `_TT`
`server.py`	`TEXT_DOCUMENT_SEMANTIC_TOKENS_FULL`; passes cached `doc.ast`

Attribute columns prefer `end_col_offset - len(attr)` when present; otherwise parent `col_offset` (approximate).

TextMate from `vscode-extension/syntaxes/pinescript.tmLanguage.json` still colors keywords, strings, and comments — the visitor does not emit those yet even though the legend reserves the slots.

Invariants and edge cases

1. **Never throws** — missing source, `tree=None`, or parse exception yields empty `data`.
2. **No range provider** — clients must not request `semanticTokens/range`.
3. **Legend is fixed** at initialize; changing indices later would break cached clients.
4. Empty `data` is valid protocol (empty file or parse failure), not an error.
5. Tokens are sorted by `(line, col)` before delta encoding.

Worked example

```
//@version=6
indicator("tok")
length = 14
plot(ta.sma(close, length))
```

Expect tokens for `length` (variable/declaration), `ta` (namespace/defaultLibrary), `sma` (method/defaultLibrary). Keywords such as `indicator` remain TextMate-only.

Failure modes

Symptom	Cause
Semantic highlighting off despite tokens	Client `editor.semanticHighlighting.enabled` false
Colors look “shifted” after a legend edit	Client cached old type indices — restart the language client
Client errors on range request	`range=False` in capabilities; client should not call it
Attribute token sits on the wrong column	Missing `end_col_offset` on the AST node — approximate path

See also

- VS Code extension — TextMate grammar
- Architecture
- Inlay hints — complementary visual annotations

INLAY HINTS

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

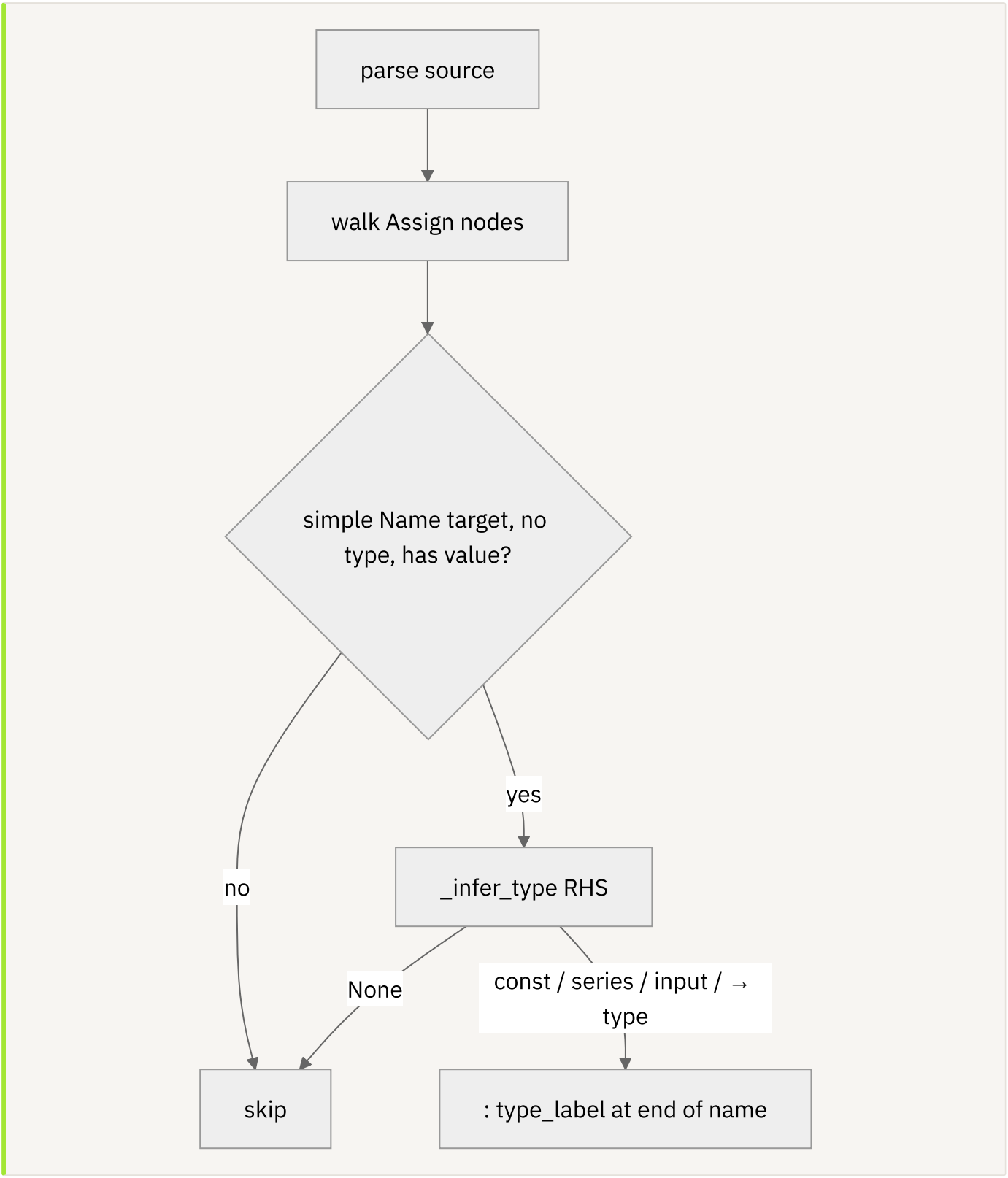
title: "Inlay Hints" description: "Inferred type annotations on simple assignments — const, series, and input kinds."

INLAY HINTS

Abstract

Inlay hints surface **inferred types** next to simple `name = expr` assignments when no explicit type annotation is present. Inference is deliberately shallow: constants, bare series builtins (`close` , ...), `input.*` constructors, and `ta` / `math` / `str` / `color` calls whose metadata `detail` contains a `→` return type. Ambiguous operators are skipped rather than guessed wrong.

Conceptual model



Example mental model (from module docstring):

```
length = 14                 →  length: const int
rsi = ta.rsi(close, 14)     →  rsi: series float   (from metadata detail)
n = input.int(5, "n")       →  n: input int
```

Interface surface

- Method: `textDocument/inlayHint`
- Capability: `InlayHintOptions(resolve_provider=False)`
- Handler: `features/inlay_hints.py` → `handle_inlay_hints` (receives cached `tree=doc.ast`)

Each hint:

Field	Value
`position`	End of target identifier (`lineno-1`, `col_offset + len(id)`)
`label`	`": {type_label}"`
`kind`	`InlayHintKind.Type`
`tooltip`	`Inferred type: {type_label}`

Returns `[]` for empty source, `None` if parse fails (client typically treats as no hints).

Internals

Constant mapping

Python value	Label
`bool`	`const bool`
`int`	`const int`
`float`	`const float`
`str`	`const string`

Builtin bare names

`open`, `high`, `low`, `close`, `volume`, `hl2`, `hlc3`, `ohlc4` → `series float`; time / bar index vars → `series int`; `na` → `na`.

Calls

- `input.{int,float,bool,string,color,symbol,session,source,time,timeframe,price}` → `input {attr}`
- `ta|math|str|color.{attr}` → parse `detail` after `→`, else fallback `"series"`

Walk

Recursive over `_fields` (same pattern as workspace symbol collection) — not limited to script top-level; nested function assigns can receive hints.

Path	Role
`features/inlay_hints.py`	Collection + inference
`providers/builtin_metadata.py`	Return-type detail for modules

Invariants and edge cases

1. **Explicit `name: type = …` suppresses hints** (`node.type is not None`).
2. **Tuple / attribute targets** are ignored.
3. **BinOp / Compare / Conditional** return `None` — no speculative arithmetic types.
4. **Metadata quality bounds accuracy** — missing `→` in detail yields coarse `series`.
5. **No resolve step** — tooltips are final.

Worked example

```
//@version=6
indicator("hints")
length = 14
src = close
avg = ta.sma(src, length)
period = input.int(20, "Period")
```

Expected hints (when parse + metadata succeed):

- `length` → `: const int`
- `src` → `: series float`
- `avg` → type from `ta.sma` detail (typically series float) or `series`
- `period` → `: input int`

`avg` will not get a hint derived from `length`'s type through the call graph — only the call's known return shape.

Failure modes

Symptom	Cause
No hints on file	Parse error → `None`
`ta.*` always `: series`	Metadata detail lacks `→` segment
Hints mid-identifier	Incorrect `col_offset` on AST node — builder issue

See also

- Completion
- Builtin metadata
- Type system

BUILTIN METADATA

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "Builtin Metadata" description: "JSON catalog for LSP completion and hover; generation pipeline; Fernet encryption, CRYPTO_KEY / METADATA_KEY, and decrypt path."

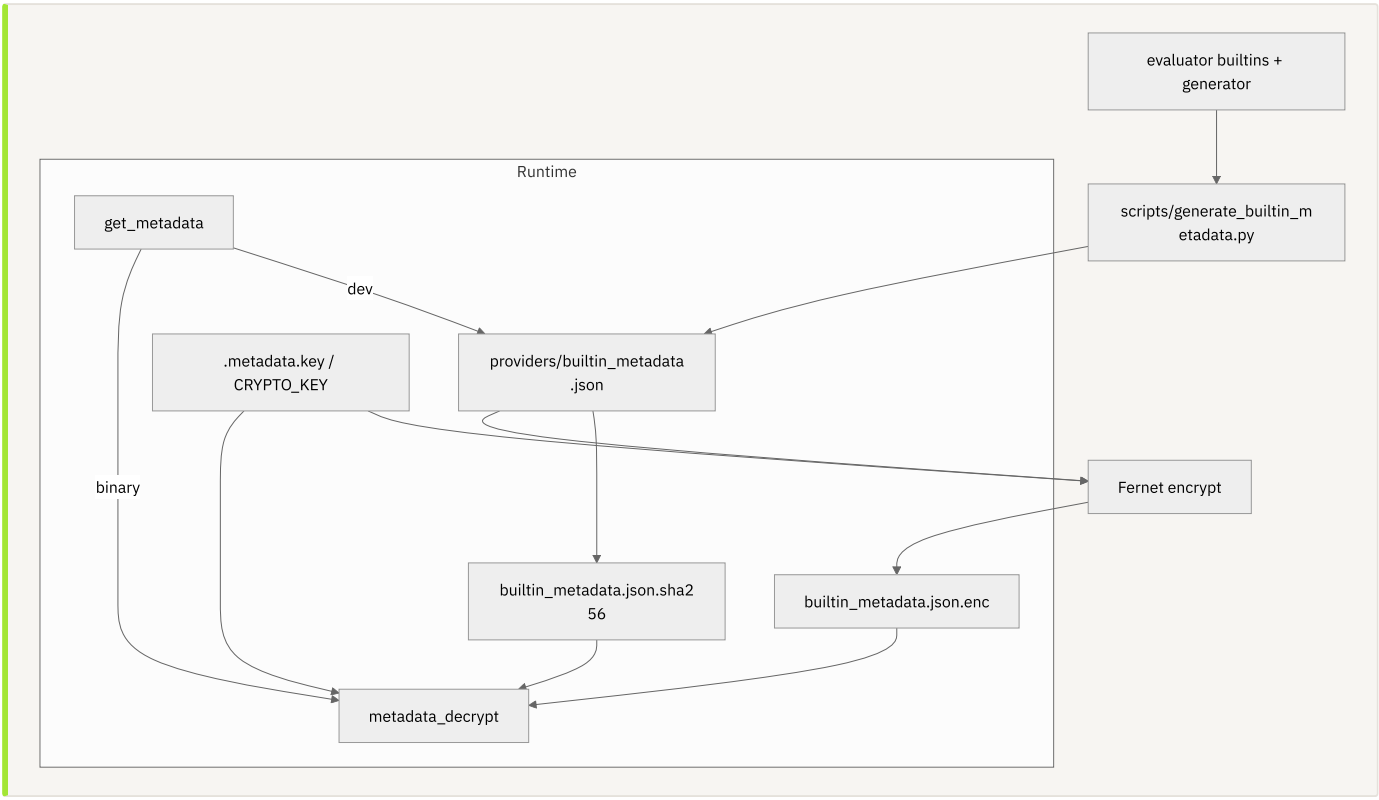
BUILTIN METADATA

Abstract

Builtin metadata is the **LSP documentation corpus**: a map from fully qualified names (`ta.sma`, `strategy.entry`, ...) to labels, signatures, briefs, snippets, and categories. It is generated from code (not hand-edited as source of truth), loaded by the language server for completion/hover/inlay, and optionally **Fernet-encrypted** for Nuitka onefile distribution so casual string extraction does not dump the full catalog.

This is obscurity for distribution hygiene, not a cryptographic access-control boundary.

Conceptual model



Interface surface (metadata record)

Typical entry in `builtin_metadata.json`:

```
{
  "ta.sma": {
    "label": "ta.sma",
    "kind": "function",
    "detail": "ta.sma(series, int) → series float",
    "brief": "…",
    "documentation": "…",
    "snippet": "ta.sma(${1:param1}, ${2:param2})",
    "category": "ta.technical_analysis"
  }
}
```

Categories drive completion headers (`ta.technical_analysis` → “Technical Analysis (ta.*)”, `builtin` → “Built-in Variables”, etc.).

Loader API

`providers/builtin_metadata.py`:

Function	Behavior
`get_metadata()`	Singleton cache; **plaintext JSON first**, else `.enc` decrypt, else `{}`
`get_builtin(name)`	Single entry or `None`
`get_builtins_by_category` / `get_all_categories`	Category queries
`fuzzy_filter`	Scored substring match for completion

Consumers

- Completion list / module / resolve
- Hover cards
- Inlay return-type extraction from `detail` after `→`

Internals

Path	Role
`scripts/generate_builtin_metadata.py`	Introspect builtins → JSON
`src/pynescript/langserver/providers/builtin_metadata.json`	Dev plaintext
`…/builtin_metadata.json.enc`	Encrypted blob for release
`…/builtin_metadata.json.sha256`	First 16 hex chars of SHA-256 of **plaintext**
`…/metadata_decrypt.py`	Fernet load + integrity check
`scripts/build/compile.py`	`encrypt_metadata()`, Nuitka include of providers dir
`scripts/build/ci_build.py`	`stage_metadata()` for CI
`scripts/build/.metadata.key`	**Gitignored** Fernet key material

Generation

After adding evaluator builtins:

```
python scripts/generate_builtin_metadata.py
```

Do not hand-edit the JSON as the long-term source of truth; regenerate from code.

Encryption (build)

`scripts/build/compile.py` → `encrypt_metadata()` / `_resolve_fernet_key()`:

1. Require `cryptography` and plaintext JSON.
2. Resolve key (first hit wins): env `CRYPTO_KEY` → `PYNESCRIPT_METADATA_KEY` → `METADATA_KEY` → existing `scripts/build/.metadata.key` → generate a new Fernet key and write that file (mode `0o600`).
3. Encrypt → `builtin_metadata.json.enc`.
4. Write truncated SHA-256 of plaintext to `.sha256`.

A random key is generated **only on first local encrypt** when no env/file key exists. For **byte-stable** encrypted artifacts across CI runs, supply a stable secret:

Context	Mechanism
GitHub Actions	`secrets.METADATA_KEY` → env (documented as `CRYPTO_KEY` in workflows / AGENTS)
Cloud Build	substitution `${_METADATA_KEY}` → `CRYPTO_KEY`
Runtime decrypt	File `.metadata.key` next to providers **or** env `PYNESCRIPT_METADATA_KEY`

Without a stable key, each CI encrypt produces a different `.enc` blob — not a functional bug, but noisy diffs and cache misses.

Decrypt path

`metadata_decrypt.py`:

1. Resolve key: PyInstaller/Nuitka data dir `.metadata.key`, else providers dir, else `PYNESCRIPT_METADATA_KEY`.
2. Fernet-decrypt `.enc`.
3. If `.sha256` present, compare `sha256(plaintext).hexdigest()[:16]`.
4. `json.loads` → dict.

Both `get_metadata()` and `get_metadata_cached()` prefer **plaintext** when both artifacts exist (developer-friendly). Difference: `get_metadata()` returns `{}` on total failure; `get_metadata_cached()` raises `FileNotFoundError` if neither file is present.

Invariants and edge cases

1. **Regenerate after builtin changes** or completion/hover/inlay drift.
2. **Empty dict on failure** fails soft (no completions) rather than crashing the server.
3. **Integrity mismatch** raises in decrypt — compiled binary should not silently serve corrupt docs.
4. **Key not in git** — never commit `.metadata.key`.
5. **Fernet is symmetric** — anyone with the binary + key material can recover plaintext; design goal is casual reverse-engineering friction and future paid-metadata swap hooks.

Worked example — local dev

```
python scripts/generate_builtin_metadata.py
# edit-free check
python -c "from pynescript.langserver.providers.builtin_metadata import get_builtin; print(get_buil
```

Worked example — encrypted binary path

```
# build (encrypts + Nuitka) — see DevOps
python scripts/build/compile.py

# or embed .metadata.key in the onefile data dir during compile
```

Failure modes

Symptom	Cause
Completions empty in release binary	Missing key / wrong `PYNESCRIPT_METADATA_KEY`
`Metadata integrity check failed`	`.enc` and `.sha256` out of sync
CI churn on `.enc`	New random key each build without `CRYPTO_KEY` / `METADATA_KEY`
Stale docs for new `ta.*`	Forgot to re-run generator

See also

- Metadata crypto (DevOps)
- Nuitka build
- Completion
- Hover

VSCODE EXTENSION

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

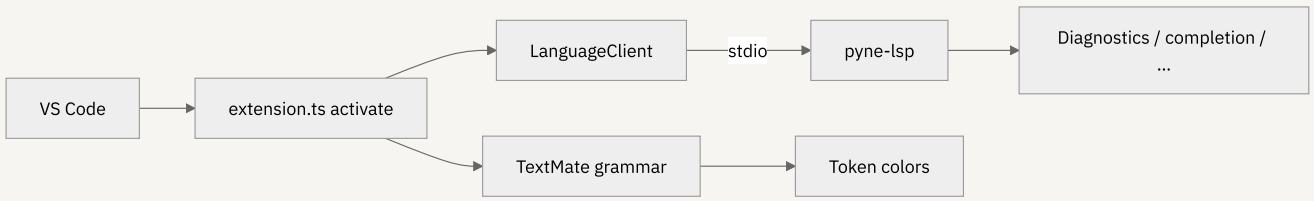
title: "VS Code Extension" description: "PYNE VS Code extension: HOOX branding, TextMate grammar, LanguageClient wiring, settings, and VSIX build."

VS CODE EXTENSION

Abstract

PYNE (`hoox-sh.pyne` **0.4.4**) is a **LanguageClient host** plus a rich TextMate grammar, branded as part of the **HOOX open trading stack**. It does not reimplement diagnostics or completion in TypeScript; it resolves `pyne-lsp` (then alias `pynescript-lsp`, then `python -m pynescript.langserver`) or a configured command over STDIO via `vscode-languageclient`, maps **`.pyne` (first-class)** plus `*.pine` / `*.pinev5` / `*.pinev6` / `*.pinescript` to language id `pinescript`, and exposes a small settings surface for enablement and feature toggles.

Conceptual model



Interface surface

Package identity

From `vscode-extension/package.json`:

Field	Value
name	`pyne`
displayName	`PYNE Language Support`
version	`0.4.4`
publisher	`hoox-sh`
extension id	`hoox-sh.pyne`
icon	`media/icon.png` (HOOX mark)
engines.vscode	`^1.91.0`
main	`./out/extension.js` (esbuild bundle)
dependency	`vscode-languageclient` ^10
galleryBanner	`#050505` dark

Language contribution

- **id:** `pinescript`
- **aliases:** `PYNE`, `pyne`, `pinescript`, `Pine`
- **extensions:** **`.pyne`** (first), `.pine`, `.pinev5`, `.pinev6`, `.pinescript`
- **configuration:** `language-configuration.json`
- **grammar:** `syntaxes/pinescript.tmLanguage.json` (`scopeName: source.pinescript`) — namespaces, annotations, hex colors, UDT/enum, multiline strings, plot/strategy builtins

Activation

```
onLanguage:pinescript
workspaceContains:**/*.pyne
workspaceContains:**/*.pine
workspaceContains:**/*.pinev5
workspaceContains:**/*.pinev6
```

Settings (`pynescript.*`)

Key	Default	Meaning
`lsp.enabled`	`true`	Skip activation when false
`lsp.command`	`auto`	`auto` = PATH `pyne-lsp` → `pynescript-lsp` → `python -m pynescript.langserver` . Or an absolute path. `"docker"` plus `lsp.args` can launch `ghcr.io/hoox-sh/pyne/lsp` (`${workspaceFolder}` is expanded).
`lsp.python`	`python3`	Interpreter for module launch when neither binary is on PATH (`pip install "hoox-pyne[lsp]"`)
`lsp.args`	`[]`	Extra server args
`formatting.enabled`	`true`	Passed as init option
`diagnostics.enabled`	`true`	Passed as init option
`completion.snippets`	`true`	Passed as init option

Initialization options object:

```
{
  "formattingEnabled": true,
  "snippetsEnabled": true,
  "diagnosticsEnabled": true
}
```

The Python server does **not** read these flags today. They are client-side documentation for hosts; capability advertisement is unconditional in `config.py` .

Commands

Command palette	ID	Action
PYNE: Restart Language Server	`pynescript.restartServer`	Stop then start the language client
PYNE: Format Document	`pynescript.formatDocument`	`editor.action.formatDocument` (`.pyne` / `.pine`)
PYNE: Convert to Pine v6	`pynescript.convertToV6`	Rewrite v1-v5 source toward v6 (LSP command or local Python)
PYNE: Show Language Server Output	`pynescript.showLspOutput`	Open Output channel / status-bar target
PYNE: Show Resolved LSP Launch Command	`pynescript.showLspCommand`	Show and copy resolved `pyne-lsp` launch

Client options

- Document selector: `pinescript` for `file` and `untitled` schemes
- File watcher: `**/*.{pyne,pine,pinev5,pinev6,pinescript}`
- Diagnostic collection name: `pynescript`
- Default formatter id: `hoox-sh.pyne`

Internals

Path	Role
`vscode-extension/src/extension.ts`	Activate / deactivate / client
`vscode-extension/package.json`	Contributes + scripts
`vscode-extension/syntaxes/pinescript.tmLanguage.json`	Grammar
`vscode-extension/language-configuration.json`	Brackets / comments
`out/extension.js`	Compiled JS

Server launch (`resolveLspLaunch` in `extension.ts`):

1. If `pynescript.lsp.command` is not `auto` , spawn that command with `lsp.args` .
2. Else try `pyne-lsp` , then `pynescript-lsp` , on `PATH` .
3. Else try `pynescript.lsp.python` (then `python3` / `python`) with `-m pynescript.langserver` .
4. Else status-bar error: install `hoox-pyne[lsp]` .

No `--parent-dir` / `PYTHONPATH` rewrite. Transport is STDIO. Release packaging expects `pyne-lsp` on `PATH` (pip, Nuitka onefile, or `docker run -i ghcr.io/hoox-sh/pyne/lsp:0.4.4`).

Build

```
cd vscode-extension
npm ci
npm run compile          # esbuild.mjs → out/extension.js
npm run package          # pyne-vscode-0.4.4.vsix
# or monorepo:
make build-vscode
```

Requires Node 22. `compile:tsc` is typecheck-only (`tsc --noEmit`).

Site download: https://hoox.sh/pyne-vscode.vsix (unversioned alias). Versioned `pyne-vscode-0.4.4.vsix` is built in-tree (`vscode-extension/`) and on GitHub Releases.

Invariants and edge cases

1. **`lsp.enabled === false` short-circuits activate** — no client, no restart command registration beyond early return (commands not registered).
2. **Binary vs module:** production users should install `hoox-pyne[lsp]` or the Nuitka onefile (`pynescript-lsp-*` on GitHub Releases) and set `pynescript.lsp.command` if not on PATH.
3. **Grammar works without LSP** — open a `.pyne` or `.pine` file offline and still get TextMate highlighting.
4. **`.pyne` is first-class** in `package.json` extensions + activation; legacy `.pine*` / `.pinescript` remain fully supported.
5. File watcher covers every associated suffix (`pyne` , `pine` , `pinev5` , `pinev6` , `pinescript`).

Worked example — extension development

```
pip install -e ".[lsp]"
cd vscode-extension && npm ci && npm run compile
# Launch Extension Development Host pointing at vscode-extension
```

Open a fixture `.pine` file; confirm Problems panel shows parse/lint diagnostics from the Python server.

Failure modes

Symptom	Cause
“Language client failed to start”	`pyne-lsp` missing / wrong command
No squiggles, highlighting OK	LSP disabled or server crash; check Output → PYNE Language Server
Restart does nothing	Client never started (`lsp.enabled` false)

See also

- Clients — non-VS Code hosts
- Architecture
- Nuitka build
- Editors guide

CLIENTS

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "Editor Clients" description: "Wire pyne-lsp into Neovim, Zed, Emacs, Helix, Sublime, and generic LSP hosts."

EDITOR CLIENTS

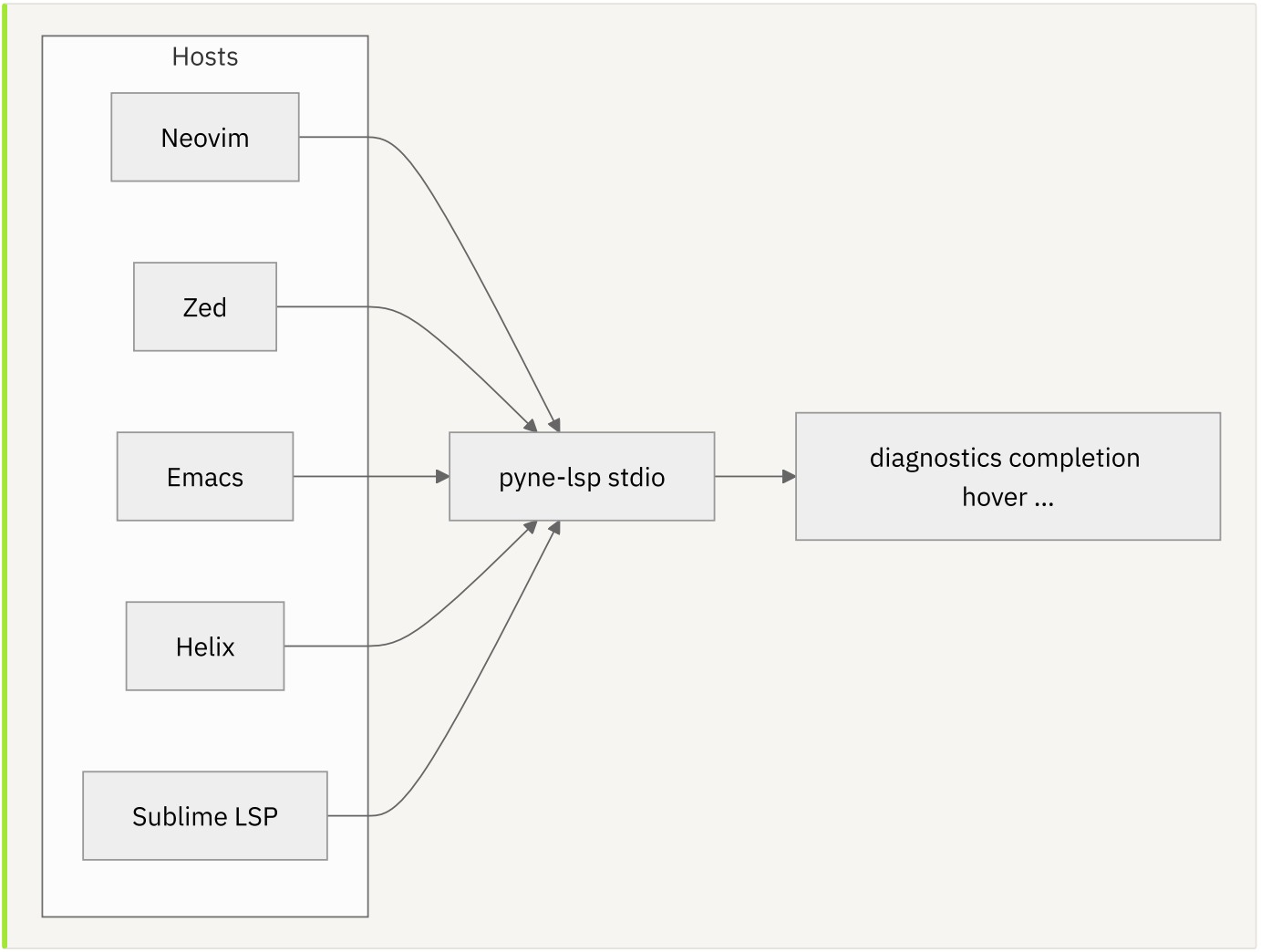
Abstract

Any editor that speaks LSP over STDIO can host PYNE intelligence. First-class snippets live in `clients/` (Neovim Lua, Zed JSON, Emacs Lisp) plus documented Helix/Sublime fragments. The VS Code path is a dedicated extension (vscode-extension); this page covers the rest.

Prerequisite: `pyne-lsp` (or `pynescript-lsp`) on `PATH` (Python 3.10+):

```
pip install "hoox-pyne[lsp]"
# development
pip install -e ".[lsp]"
```

Conceptual model



Interface surface — repo files

File	Editor
`clients/neovim.lua`	Neovim (`nvim-lspconfig` or manual)
`clients/zed.json`	Zed `settings.json` fragment
`clients/emacs.el`	Emacs `lsp-mode` + `pinescript-mode`
`clients/README.md`	Human-oriented install notes
`vscode-extension/`	VS Code / Cursor-compatible

Neovim

Return table from `clients/neovim.lua` :

```
return {
  cmd = { 'pyne-lsp' },
  filetypes = { 'pinescript' },
  root_dir = function(fname)
    return vim.fs.root(fname, { '.git', '*.pine', '*.pinev5', '*.pinev6', 'pyproject.toml' })
      or vim.fn.getcwd()
  end,
  settings = {
    pinescript = {
      formatting = { enabled = true },
      diagnostics = { enabled = true },
      completion = { snippets = true },
    },
  },
}
```

Suggested maps when attaching: `gd` definition, `gr` references, `K` hover, format via `vim.lsp.buf.format`.

Register filetype if needed:

```
au BufRead,BufNewFile *.pyne,*.pine,*.pinev5,*.pinev6,*.pinescript setfiletype pinescript
```

Zed

Merge `clients/zed.json` into `~/.config/zed/settings.json`:

```
{
  "languages": {
    "Pine Script": {
      "language_servers": ["pynescript"]
    }
  },
  "language_servers": {
    "pynescript": {
      "command": "pyne-lsp",
      "arguments": ["--stdio"],
      "languages": ["Pine Script"]
    }
  }
}
```

Zed does not ship a first-party Pine grammar in this repo. Map `*.pyne` / `*.pine` to the `"Pine Script"` language name (or whatever your grammar extension registers) so the `language_servers` key attaches.

Emacs

`clients/emacs.el` registers an `lsp-mode` client:

```
(lsp-register-client
 (make-lsp-client
  :new-connection (lsp-stdio-connection '("pyne-lsp" "--stdio"))
  :major-modes '(pinescript-mode)
  :server-id 'pynescript))
```

Defines a minimal `pinescript-mode` with font-lock keywords and `auto-mode-alist` for `\\.pine\\'` / `\\.pinev[0-9]+\\'` (add `\\.pyne\\'` locally if you use the product suffix). Optional keys: `C-c C-c` format, `M-.` definition, `M-?` references.

Helix

`~/.config/helix/languages.toml`:

```
[[language]]
name = "pinescript"
scope = "source.pinescript"
file-types = ["pyne", "pine", "pinev5", "pinev6"]
roots = ["pyproject.toml"]
command = "pyne-lsp"
args = ["--stdio"]
```

Sublime Text

With Package Control **LSP**:

```
{
  "clients": {
    "pynescript": {
      "command": ["pyne-lsp", "--stdio"],
      "selector": "source.pinescript",
      "initializationOptions": {}
    }
  }
}
```

Generic / Cursor

```
pyne-lsp
# aliases:
pynescript-lsp --stdio
# container (stdio):
docker run --rm -i -v "$PWD:/work" -w /work ghcr.io/hoox-sh/pyne/lsp:0.6.7
```

Point the host's "external language server" command at that binary. Cursor users can also load the VS Code extension (`hoox-sh.pyne`) in compatible mode when packaged as VSIX.

Internals

Server entry always uses STDIO (`start_io` in `__main__.py`). Capability negotiation is identical across hosts — differences are client UI only (how inlay hints render, whether pull diagnostics are used, etc.).

Tests: `tests/test_lsp_features.py` (handlers with fake params), `tests/test_langserver.py` (e2e with pytest-asyncio) — not editor-specific.

Invariants and edge cases

1. **Language id** should be `pinescript` for VS Code parity; some editors invent their own names — map filetypes carefully.
2. **Root directory** heuristics vary; wrong root rarely breaks single-file Pine editing (workspace is URI-keyed open buffers).
3. **Settings keys** under `pinescript` in Neovim are conventional; the Python server does not currently enforce a rich `workspace/configuration` schema for all of them.
4. Nuitka onefile can replace the module entry for air-gapped machines — same command name if installed on PATH.

Failure modes

Symptom	Cause
Server exits immediately	Missing pygls / incomplete install `.[lsp]`
Filetype never attaches	Extension not mapped to `pinescript`
Features missing in Helix	Older Helix without inlay / semantic support — grammar still useful

See also

- VS Code extension
- Architecture
- End-user editors guide